Studio Nx

AI 구현 설명 01 — 개발 환경 구축

2026-10-12

AI 구현 설명 01 — 개발 환경 구축

항목	내용
작성	Claude (AI)
작성일	2026-10-01
대상	`studionx/` 폴더 전체 (구현 계획 S1-01의 환경 부분)
원칙	기획서의 AI 규칙에 따라 **환경과 뼈대만** 만들었고, 기능 구현(JS 로직, API 기능, C++)은 하지 않았습니다

---

1. 무엇을 만들었나

파일/폴더	역할	비고
`.venv/`	Python 3.13 가상환경	PC의 기본 버전은 3.14이지만, 라이브러리 호환성(바이너리 휠)이 안정적인 3.13을 골랐습니다
`backend/requirements.txt`	실행에 필요한 라이브러리	FastAPI, uvicorn, pydantic-settings, python-multipart, SQLAlchemy, PyMySQL, cryptography, PyJWT, bcrypt, sentry-sdk
`backend/requirements-dev.txt`	개발에 필요한 라이브러리	pytest, httpx2(TestClient), ruff
`backend/pyproject.toml`	pytest와 ruff 설정	
`backend/app/main.py`	FastAPI 진입점	CORS, `/api/v1` 라우터, 프론트 정적 파일 마운트, Sentry 초기화를 합니다
`backend/app/core/config.py`	설정	`.env`를 읽고, DB 접속 URL을 조립합니다
`backend/app/core/sentry.py`	Sentry 초기화	DSN이 없으면 끕니다. 비밀번호와 토큰 키는 `[Filtered]`로 가립니다
`backend/app/db/session.py`	DB 엔진과 세션	`Base`(ORM 부모 클래스)와 `get_db`(라우터 의존성)가 있습니다
`backend/app/api/v1/router.py`	라우터를 등록하는 곳	환경 점검용 `/health`와 `/health/db`만 있습니다
`backend/app/{models,schemas,services,workers}/`	빈 패키지	기능을 구현할 때 사용자가 채웁니다
`backend/tests/test_health.py`	환경 확인 테스트	
`frontend/static/js/core/sentry-init.js`	프론트 Sentry **템플릿**	함수 시그니처와 TODO만 있습니다 (JS 구현은 사용자가 합니다)
`db/00_create_database.sql`	DB와 계정을 만드는 SQL	**아직 실행하지 않았습니다** (root 비밀번호가 필요합니다)
`native/README.md`	C++ 영역 안내	코드는 없습니다 (사용자 영역)
`.env.example`, `.env`	환경 변수	`.env`는 git에서 제외합니다
`.gitignore`, `.editorconfig`, `README.md`	프로젝트 공통 설정	

2. 왜 이렇게 했나

2.1 프론트엔드를 FastAPI가 서빙

- 기술 스택이 Vanilla JS(빌드 도구 없음)이므로 별도의 개발 서버가 필요하지 않습니다.
- 같은 주소(localhost:8000)에서 API와 페이지를 함께 제공하면 **CORS와 쿠키 문제가 없어서** 리프레시 토큰(HttpOnly 쿠키, PRD 01 FR-105)을 쓰기 쉽습니다.
- `app.mount("/", StaticFiles(...))`를 **라우터 등록 뒤**에 둔 이유: 먼저 마운트하면 `/api/...` 요청까지 정적 파일 쪽으로 가기 때문입니다.

2.2 SQLAlchemy 2 + PyMySQL (동기 방식)

- 비동기 드라이버(aiomysql)보다 구조가 단순하고 디버깅이 쉬워서 학습과 1인 개발에 알맞습니다. FastAPI는 동기 함수 라우터를 스레드풀에서 실행하므로 초기에는 성능 문제도 없습니다.
- `cryptography`는 PyMySQL이 MySQL 8의 기본 인증 방식(caching_sha2_password)을 처리하는 데 필요합니다.
- `pool_pre_ping=True`: 오래 쉰 연결이 끊겨 있으면 자동으로 다시 연결합니다.

2.3 설정은 `.env` + pydantic-settings

- 비밀값(DB 비밀번호, JWT 키, Sentry DSN)을 코드에 넣지 않기 위해서입니다. git에는 `.env.example`만 올라갑니다.
- 환경 변수 이름은 대소문자를 구분하지 않고 `DB_PASSWORD` → `db_password`로 연결됩니다.

2.4 Sentry

- DSN이 비어 있으면 초기화하지 않으므로, Sentry 계정 없이도 개발할 수 있습니다.
- `send_default_pii=False`로 두고 `before_send`에서 민감한 키를 지웁니다. PRD 00 §7의 "민감정보는 Sentry 전송 전 제거"를 반영한 것입니다.

2.5 DB 계정 권한

- 개발 단계에서는 앱 계정이 테이블을 만들 수 있도록 CREATE/ALTER/DROP까지 주었습니다. 운영에서는 SELECT/INSERT/UPDATE/DELETE만 남기는 것이 안전합니다 (DB.md §1.2의 원래 의도).

3. 검증 결과 (2026-10-01)

확인	결과
`pip install -r requirements-dev.txt`	성공했습니다 (FastAPI 0.142, SQLAlchemy 2.1, sentry-sdk 2.71 등)
`pytest`	1 passed이고, 경고는 없습니다
`ruff check`	All checks passed
`GET /api/v1/health`	`{"status":"ok"}`
`GET /docs`	200
`GET /static/js/core/sentry-init.js`	200 (정적 파일 서빙을 확인했습니다)
`GET /api/v1/health/db`	`OperationalError`이며, **정상**입니다. DB와 계정을 아직 만들지 않았기 때문입니다

4. 남은 일 (사용자)

1. `db/00_create_database.sql`의 `<APP_PASSWORD>`를 바꾸고 `mysql -u root -p -e "source db/00_create_database.sql"`을 실행합니다 (PowerShell은 `<`를 지원하지 않으므로 `source`를 씁니다).
2. 같은 비밀번호를 `studionx/.env`의 `DB_PASSWORD`에, 무작위 문자열을 `JWT_SECRET`에 넣습니다.
3. `/api/v1/health/db`가 `{"db":"ok"}`인지 확인합니다.
4. (선택) Sentry 프로젝트를 만들고 `SENTRY_DSN`을 넣습니다.
5. (C++) CMake, Emscripten, JUCE를 설치합니다 (`native/README.md` 참고).

5. 자주 받을 질문 대비

- **Q. 왜 Django가 아니라 FastAPI인가요?** 기획서의 기술 스택입니다. 자동 API 문서(/docs), Pydantic 검증, WebSocket 지원(Phase 6)이 장점입니다.
- **Q. 왜 프론트 빌드 도구(Vite 등)를 쓰지 않았나요?** 기술 스택이 Vanilla JS이고, ES Module(`<script type="module">`)만으로도 모듈을 나눌 수 있기 때문입니다. AudioWorklet도 별도 파일로 그대로 불러올 수 있습니다.
- **Q. 테이블은 왜 아직 없나요?** 기획서 원칙상 DB는 새 database에 만들되, 테이블은 Phase 1 작업(S1-02)에서 DB.md §5의 DDL로 만들 예정입니다.